

一个人 + AI 的 22 天

KOLMatrix 与 Triad Workflow

Solo dev 如何用多 agent 框架交付企业级 SaaS

主讲人 | 2026-05 | `kol.guangai.ai`

22 天 + Solo dev + 企业级 SaaS = ?

难题 1 · 体量

5 国语言 · 4 平台数据集成
AI 邮件生成 · RLS 多租户
传统估算：4-5 人 × 3 个月

难题 2 · AI 单独失控

- 让 AI 自评 = 失控
- AI 越权改架构 = 失控
- 上下文塞满 = 丢状态

难题 3 · 多 AI 无方法论

agent 间互通信
容易陷入幻觉循环
现成框架几乎没有

18 分钟 · 4 个问题

#	问题	时长	章节
1	我做了什么？	2 min	产品速览
2	怎么做到的？	5 min	Triad Workflow
3	验证了什么？	3 min	实战数据 + 教训
4	你能拿走什么？	4 min	可复用经验

KOLMatrix

全球游戏 KOL/KOC 智能营销管理平台

Discover

KOL 发现
4 平台数据

Evaluate

engagement
评估打分

Contact

AI 个性化
邮件外联

Track

Campaign
进度跟踪

5 国语言 (CN/EN/JA/KO/ES) · 4 平台 (YT/TikTok/IG/X)

多租户 RLS · **已上线** `kol.guangai.ai`

22 天体量数据

982

commits
平均 45/天

61K

行 TypeScript
473 个文件

82

份 specs
规格驱动

14

份 ADR
架构决策

103

份验收报告
全部留痕

30+

framework 版本
迭代演进

大部分代码 / 规格 / 报告由 AI agent 生成；我的角色是设计、决策、监督

Triad Workflow

核心思想 · 四要素

三角色不重叠

Planner / Generator / Evaluator

无人评估自己的工作

→ 解决「AI 自评失控」

状态机驱动

7 状态硬约束

不在你的状态就退出

→ 解决「AI 越权失控」

Git 作协作总线

`progress.json` + `.auto-memory/`

异步交接, 跨设备/工具

→ 解决「多 AI 协作无方法论」

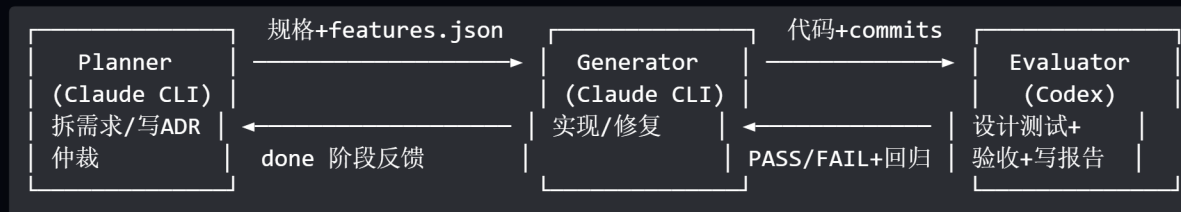
记忆分层

T0 / T1 / T2 确定性加载

避免上下文塞满丢状态

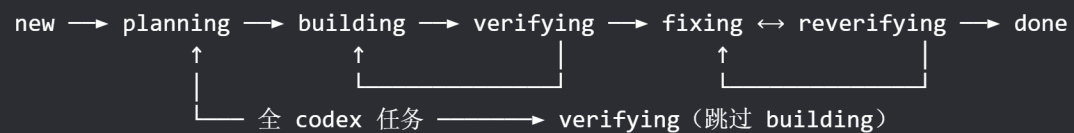
→ 解决「上下文失控」

三角色协作



铁律： 测试设计权 100% 归 Evaluator · Generator 不准写自己的测试
Planner 不准改产品代码（铁律 9：紧急 hotfix 也走流程）

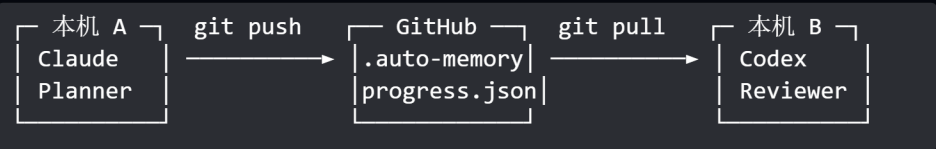
状态机 • 7 状态硬约束



关键： AI agent 启动第一件事就是读 `progress.json` 的 `status` —— 不匹配自己角色就直接退出。
不靠它「自觉」，是状态机硬不让它干。

记忆分层 + 跨设备协作

层	何时加载	文件	大小
T0	每次启动必读	MEMORY.md + project-status.md + environment.md	≤ 30 行
T1	按当前角色加载	role-context/{角色}.md	≤ 50 行
T2	触发条件命中	feedback-*.md / reference-*.md	按需



早上台式机当 Planner，下午笔记本当 Reviewer，状态完全无缝

22 天实战 KPI

交付密度

982 commits / 22 天
平均 **45**/天
高峰单日 **78** commits

验收质量

103 份验收报告全部留痕
14 份 ADR 长期决策
fix_round 极少 > 2

时间分布

42 个独立批次
平均批次 4-6 小时
上线 buffer 3 天

关键洞察： 框架强制 Evaluator 拥有完整测试设计权 → Generator 写代码时主动反推「这功能怎么验收」 → spec 质量被前置推高 → 后期 fix 反而减少。**严格分工 ≠ 低效。**

30+ 框架版本 = 30+ 次实战教训

版本	沉淀的规则	起因（真实事故）
v0.5.0	共享记忆分层 T0/T1/T2	上下文塞满丢状态事故
v0.7.1	Spec 起草前必 grep 实物	字面假设导致回归
v0.9.0	Pre-Impl Audit 模式	Generator 开工后才发现 spec 错
v0.9.4	铁律 #10: commit-tag 必对应 feature	Planner docs commit 误带 9 文件 WIP 入 main
v0.9.11	状态机 JSON 必 <code>json.load</code> 校验	progress.json 缺一个 <code>}</code> 入 main
v0.9.18	auth role enum 实物核查	字面 <code>'admin'</code> vs 真实 <code>'tenant_admin'</code>
v0.9.19	External API zod schema 实物 sample	单测 mock PASS 但 prod 50/41 row 触发 parse error

诚实点： 这些都是踩出来的，不是设计出来的

工程层 7 条经验（给工程师）

经验	Why
设计系统先行	第一个批次必须是设计系统，不是业务页面。后期返工成本是前期 5x
Pre-Impl Audit	Generator 开工前主动审计 spec vs 实物 → Planner 裁决
CI 红不开新功能	每次 push 后必查 <code>gh run</code> ，红色立即停手
Critical 修必带回归	修复同 commit 补 regression test，否则 bug 一定回归
单功能单 migration	不打包 schema 变更，回滚粒度对应风险粒度
视觉规范 $\pm 2\text{px}$ / $\Delta E < 2$	pixel-perfect 量化标准；模糊标准 = AI 输出不收敛
ADR 沉淀关键决策	AI 长期协作需要决策"长期记忆"，本项目 14 份

AI 协作层 5 条经验（人人受用 · 1/2）

1 · 不让 AI 评估自己

验收必须由不同角色 / **不同工具** 完成
Generator 用 Claude, Evaluator
用 Codex
强化职责边界

2 · 状态机 > 自觉规则

文档里写「禁止 X」= 软约束
「不在你的状态就退出」= 硬约束

3 · 记忆分层 > 塞满

T0 必读 30 行 + T1 角色 50 行 +
T2 触发加载
每次启动加载越小，焦点越清晰

AI 协作层 5 条经验 (人人受用 · 2/2)

4 • Spec 前必 grep 实物

字面假设 / mock shape / 旧记忆 = 三大盲区
auth enum / 真数据 sample / git log 必查
v0.9.14 后框架硬性入口检查

5 • Git 是协作总线

`progress.json` + `.auto-memory/` 入 git
→ 跨设备 / 跨工具协作免费实现
`.agent-id` 不入 git → 本机身份隔离

→ **Triad Workflow 已开源为 template repo**

一个人 + AI 不是 demo 工具

而是企业级生产力

前提：方法论 · 约束 · 沉淀

产品

`kol.guangai.ai`

5 国语言 / 4 平台 / RLS

框架（可复用）

`tripplemay/harness-template`

`npx degit ... && bash bootstrap.sh`

Q & A